

Retrieval Stuff, NLP Course, Lecture 9

Michael Chertushkin

Plan for the lecture

- LLMs are perfect hallucinators
- Information Retrieval and Relevance Gap
- Naive RAG
- How to train Naive RAG
- Advanced RAG
- Iterative RAG
- Rag is Dead ©
- Future of retrieval and agentic RAG

What is Hallucination?

- Hallucination is phenomenon when LLMs respond with some text that “look” correct, but is not correct
- Hallucinations are not very dangerous in general B2C use-cases (except certain specific domains like health/finance advisory)
- Hallucinations are considered the main showstopper for B2B market - reason why we have not heard a lot of Gen AI adoption across B2B is because of hallucinations
- But why?

What is Hallucination?

- Hallucination is when a model generates text that “look” correct, but is not based on the input data.
- Hallucinations are common in many cases (except in certain specific cases).
- Hallucinations are a major problem in the B market - reason why we have many issues is because of hallucinations.
- But why?



Why Hallucinations happen?

- Let's take a step back and look into how LLMs are trained

GPT Assistant training pipeline

Stage	Pretraining	Supervised Finetuning	Reward Modeling	Reinforcement Learning
Dataset	Raw internet text trillions of words low-quality, large quantity	Demonstrations Ideal Assistant responses, ~10-100K (prompt, response) written by contractors low quantity, high quality	Comparisons 100K –1M comparisons written by contractors low quantity, high quality	Prompts ~10K-100K prompts written by contractors low quantity, high quality
Algorithm	Language modeling predict the next token	Language modeling predict the next token	Binary classification predict rewards consistent w preferences	Reinforcement Learning generate tokens that maximize the reward
Model	Base model	SFT model	RM model	RL model
Notes	1000s of GPUs months of training ex: GPT, LLaMA, PaLM can deploy this model	1-100 GPUs days of training ex: Vicuna-13B can deploy this model	1-100 GPUs days of training	1-100 GPUs days of training ex: ChatGPT, Claude can deploy this model

Why Hallucinations happen?

- Picture on the previous slide is not “formally” correct in 2025
- in 2025 people usually decompose LLM training into two phases:
 - Pre-training
 - Post-training
- But still lets focus independently on these three different steps of LLMs

Why Hallucinations happen? During Pretraining

- Pretraining of LLMs is autoregressive ML task with:
 - CrossEntropy as objective function
 - $(X, Y) := (\text{context}, \text{token})$
- Pretraining of LLMs does not try to:
 - make the result accurate as possible
 - check against database
 - say “I don’t know”
- The very nature of minimizing CrossEntropy on next token prediction task will favor “guessing”

Why Hallucinations happen? During SFT

- SFT of LLMs is autoregressive ML task with:
 - CrossEntropy as objective function
 - $(X, Y) := (\text{instruction}, \text{instruction_result})$
- SFT of LLMs also does not try anything that can make responses “factually correct”
- Let’s look at the last phase - human alignment (post-training)

Why Hallucinations happen? During Posttraining

- Postraining of LLMs is the human alignment attempt on choosing between rejected and accepted responses
- Depending on the concrete method:
 - We use Reward Model + PPO (for RLHF) and minimize pairwise ranking loss
 - We use LLM as a reward model itself (for DPO)
- What's important is that model favors more what annotators favor more, and annotators favor more the responses that are “confident”, “sound plausible”, “go into details”
-

Why Hallucinations happen? Summary

- At the current stage of LLM development Hallucinations are incentivised at every stage of LLM training and are inevitable
- As previously was mentioned Hallucinations block the mainstream B2B adoption of Gen AI (together with security concerns and connector issues with 3-p systems)
- There are attempts to put into the loss the “penalty” for confident but wrong responses, but this is not the mainstream

What shall we do

- If we can't reduce hallucinations during LLM training (yet), maybe we can do something during test-time to mitigate them
- Let's take even larger step back and talk about something completely irrelevant (from the first glance) - **Information Retrieval** field

What shall we

- If we can't reach something due to something
- Let's take even more irrelevant (from



maybe we can do

completely

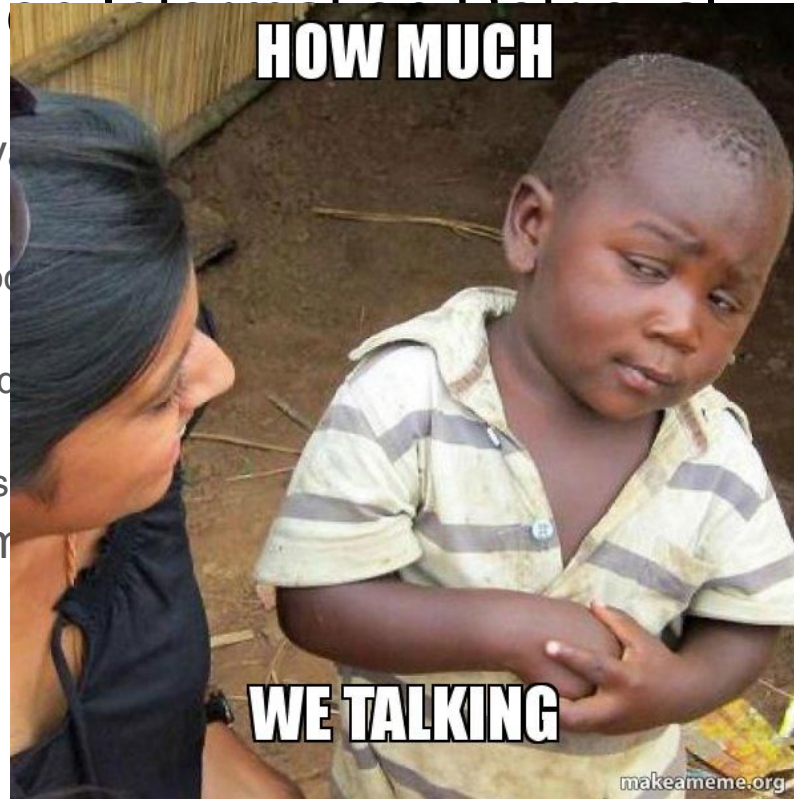


1000 feet view on Information Retrieval

- IR is to find relevant items in a large corpus, given a query
- We have:
 - Collection of documents
 - We have query
 - We want to produce ranked list $d_1, d_2, \dots, d_n$, such that most relevant docs for query were the first
 - We achieve this by learning the scoring function $s(q, d)$
- How many information retrieval methods do you know?

1000 feet view

- IR is to find relevant
- We have:
 - Collection of docs
 - We have query
 - We want to produce the first
 - We achieve this
- How many inform



query

relevant docs for query were

IR: TF-IDF (Hello from ML, Part 1)

- We take whole corpus of docs and do `TfidfVectorizer()`
- When query comes in we transform it to vector and find closest vectors
- Works very well as a baseline
- Problems:
 - Ignores order
 - Too basic
-

IT AIN'T MUCH

BUT IT'S HONEST WORK

IR: BM25 - most notable classical IR

- We take whole corpus of docs and do TfidfVectorizer()
- We score each document for (D, Q) as:

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

- Works well (they iterated 25 times - 25 is the version number)
- There is a reason why we mention BM25, we will return to it later

IR: Pros and Cons of classical methods

- Pros:

- Very fast
- Excellent for exact matches
- Interpretable (score for each term)

- Cons:

- Doctor/physician is not recognized, no synonyms
- Highly dependent on tokenization/stemming/lemmatization
- Not possible to do multi-hop reasoning

IR: Metrics

- IR is mature field with a bunch of classical metrics:
 - Precision@k
 - Recall@k
 - MRR (mean reciprocal rank)
 - NDCG@k

Now let's go back to our LLMs and hallucinations and lets pivot the idea

From IR to RAG

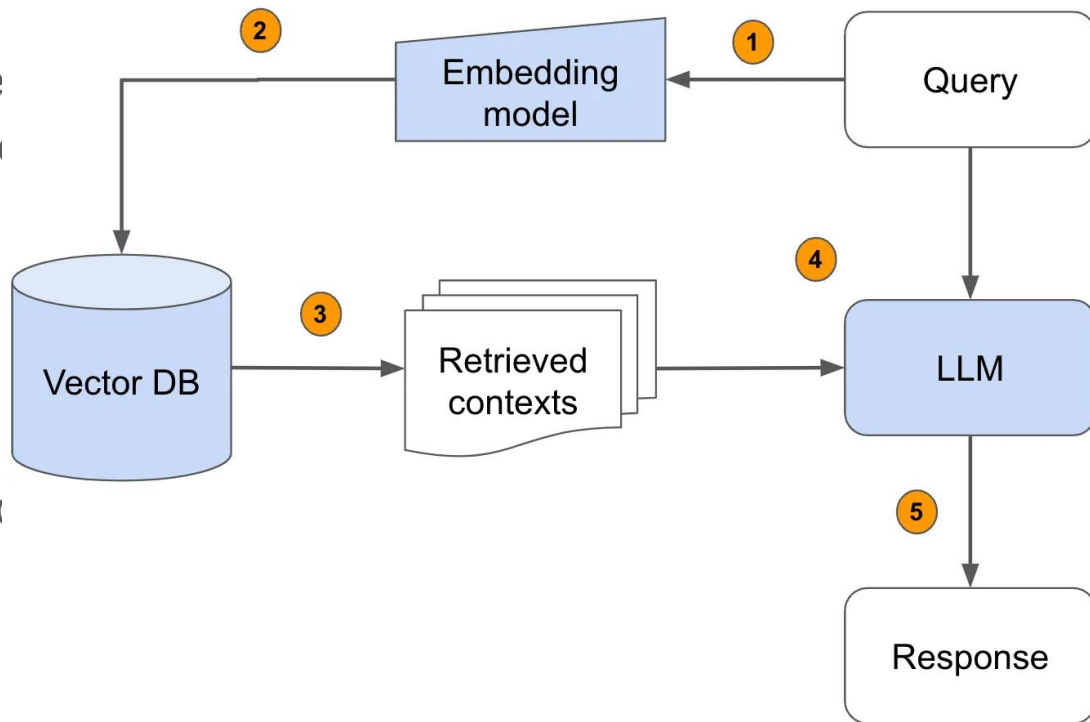
- RAG (retrieval augmented generation) is the technique where we “enrich” the context of LLM during test-time with retrieved information based on the query
- In other words, we condition on retrieved documents

$$y \sim p_{\theta}(y \mid q, d_{\pi(1)}, \dots, d_{\pi(k)})$$

- “Condition” is as simple as just appending docs into the context (appending where - in the beginning, in the middle, or in the end?)

From IR to RAG

- RAG (retrieval-augmented generation)
- In other words, we “enrich” the context used on the query



- “Condition where -

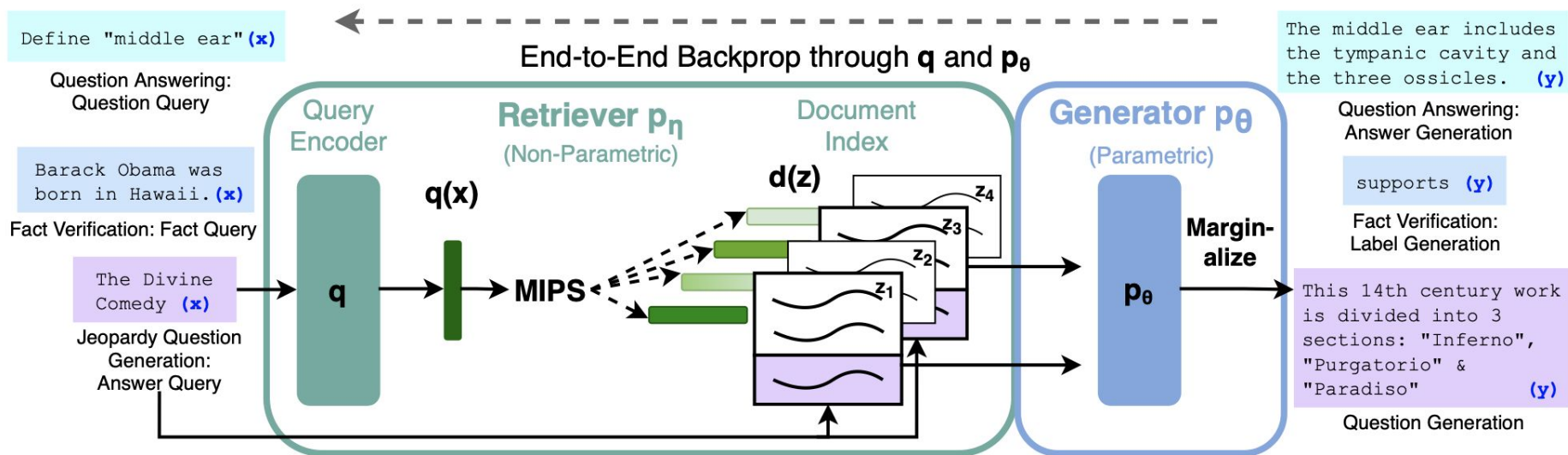
ext (appending

2 Types of RAG

- Joint RAG (RAG-Sequence / RAG-Token)
- Modular RAG

We will mostly talk about Modular RAG in the rest of the lecture, but let's quickly examine what is Joint RAG

Joint RAG (Lewis et al., 2020)



$$p(y | q) = \sum_d p_\eta(d | q) p_\theta(y | q, d)$$

Joint RAG: RAG-Sequence (Lewis et al., 2020)

- Here one document z is chosen (as a latent variable) and used to generate the entire output sequence.

$$p_{\text{RAG-Seq}}(y \mid x) \approx \sum_{z \in \text{TopK}(x)} p_{\eta}(z \mid x) p_{\theta}(y \mid x, z)$$

$$p_{\theta}(y \mid x, z) = \prod_{i=1}^N p_{\theta}(y_i \mid x, z, y_{1:i-1}).$$

$$p_{\text{RAG-Sequence}}(y \mid x) \approx \sum_{z \in \text{top-k}(p(\cdot \mid x))} p_{\eta}(z \mid x) \prod_{i=1}^N p_{\theta}(y_i \mid x, z, y_{1:i-1}).$$

Joint RAG: RAG-Token (Lewis et al., 2020)

- Now, each token y can come from a different document
-

$$p_{\text{RAG-Token}}(y \mid x) \approx \prod_{i=1}^N \sum_{z \in \text{TopK}(x)} p_{\eta}(z \mid x) p_{\theta}(y_i \mid x, z, y_{1:i-1}).$$

Joint RAG (Lewis et al., 2020)

- The idea of joint training looks really plausible and was tried in bunch of others works
- But there is a fundamental reason why Joint RAG and other similar types of research did not find mass adoption
- Anyone wants to guess why?

Modular RAG

- LLM is usually the black box (at least in industrial settings) -> there is huge demand for methods that can accurately augment the context without touching LLM
- We have retriever, which we can tune (through Embedding Tuning) and try to get high precision@k and recall@k
- We have reranker (cross-encoder), which we also can tune to get high MRR/NDCG
- This whole setup creates the problem called “relevance gap”

Relevance Gap: The problem

- The retriever is optimized to bring documents that look ‘relevant’ under some IR metric
- But the generator cares for documents that actually help it answer the question faithfully and correctly
- This gives the following problem:
 - The retriever returns documents that mention the right keywords but don’t contain the needed evidence
 - Or it returns the right doc, but buried among many near-duplicates or noisy chunks, and the LLM never focuses on the crucial bit
- The rest of the lecture we will talk how to close the relevance gap
- The “Advanced RAG” is actually various methods glued together

But before that...

- Before we go into the rabbit hole of speedrunning over various Engineering tips & hacks in the real, of Advanced RAG, let's actually understand how we can train our own baby RAG ourselves
- Why?
 - Because it's funny
 - Because we can

Training Baby RAG 101

- We need to define the ML Model (architecture of NN or choose the class of methods from some family)
- We need to define the objective/loss/error function/etc
- We need to define (X, Y) - training tuples

Lets begin with tuples

Training Baby RAG 101: Data

- We want relevant documents be close to the query in some latent space
- We want irrelevant documents be distant to the query in some latent space
- Choice is obvious:
 - We will have $(q, d, 1)$ for relevant q and d
 - We will have $(q, d, 0)$ for non-relevant q and d
- Question for attentive reader: can we use -1 for non-relevant q and d ?

Training Baby RAG 101: Contrastive Loss

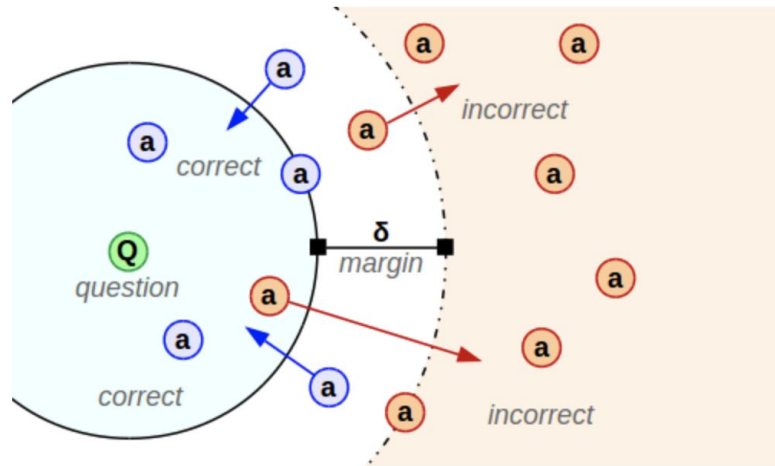
- We want some loss that will push closer relevant queries and docs
- One of the most popular used is Contrastive (Siamese) Loss (when we have 1 and 0 as labels)
- Below is the example of “Margin Based Contrastive Loss”
- D - Euclidean distance between the query embedding and the document embedding
- M - is margin

$$L(q, d, Y) = Y \cdot D^2 + (1 - Y) \cdot \max(0, M - D)^2$$

Training Baby RAG 101: Triplet Loss

- Another variant is without explicitly using labels (this is Loss from seminar)
- One of the most popular used is Triplet Loss:

$$L = \frac{1}{N} \sum_{q, a^+, a^-} \max(0, \delta - \text{sim}[V_q(q), V_a(a^+)] + \text{sim}[V_q(q), V_a(a^-)])$$



Training Baby RAG 101: Model Architecture

- We want some component that will take (Query, Doc) and produce some embedding
- Which model arch is good here? We will think about it during seminar

Three Phases of Advanced RAG

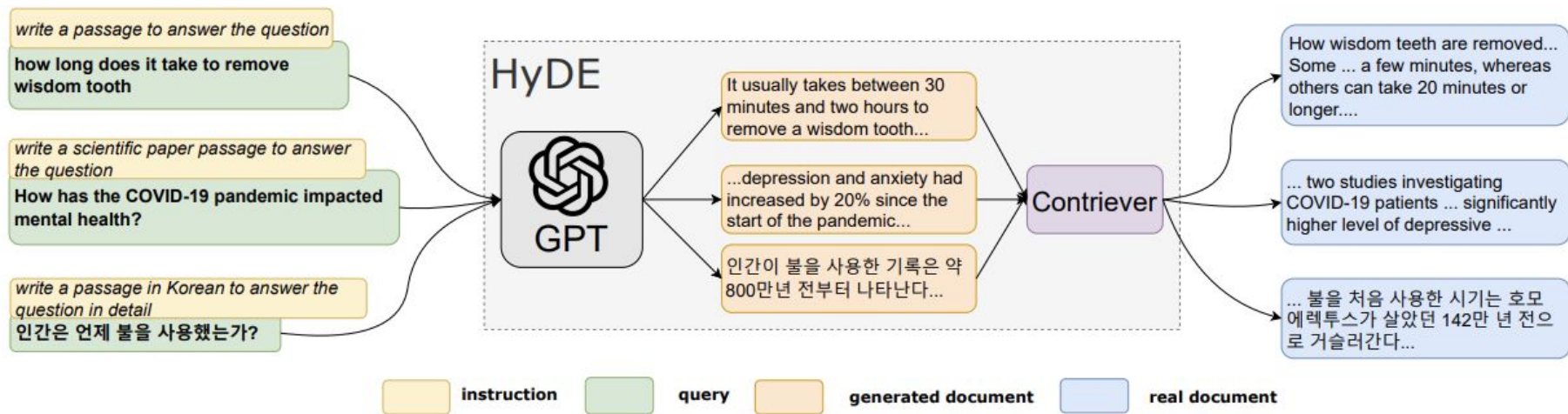
- Modern “Advanced RAG” treats the pipeline as three optimizable stages:
 - Pre-Retrieval (indexing, advanced chunking)
 - Retrieval (actual retrieval)
 - Post-Retrieval (compression, filtering, reranking)

Pre-Retrieval

- Main objective is to fix Garbage in/Garbage out
- If the index is poorly structured or query is ambiguous, no amount of retrieval or tuning can save the output
- Your RAG system isn't bad. Your queries are ©
- There are 2 main topics here:
 - Query Transformation methods (including query expansion)
 - Index & Corpus Design methods (chunking, metadata, multi-vector index)

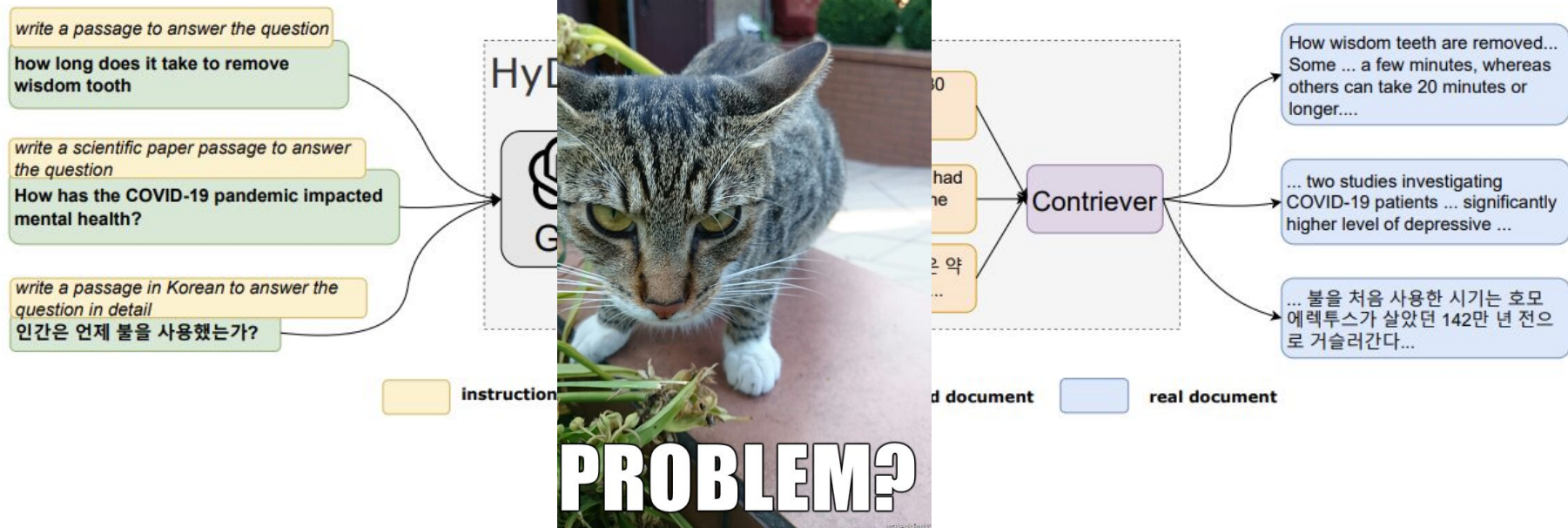
Pre-Retrieval: Query Transformations

- Use another LLM to create the doc from the query



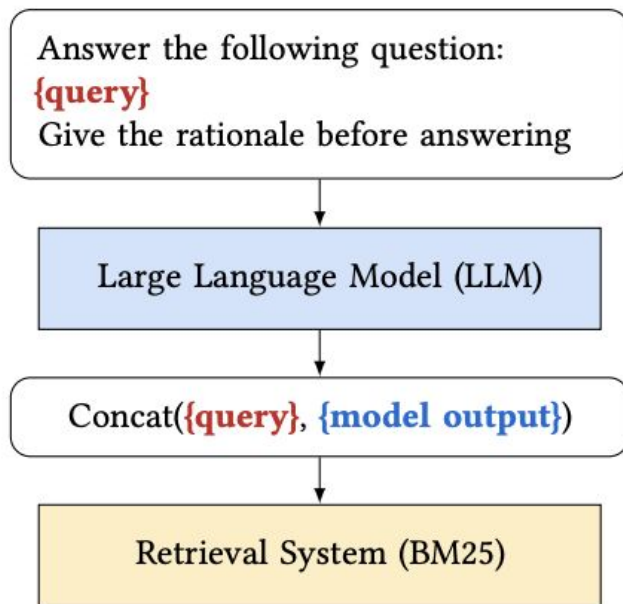
Pre-Retrieval: Query Transformations

- Use another LLM to create ANY query



Pre-Retrieval: Query Transformations

- Use another LLM to rewrite the rewrite, expand or decompose the query



Pre-Retrieval: Query Transformations

- Use another LLM to create synonyms queries and launch them all into retriever
- This greatly increases the recall but also harms the precision
- This approach is used in many production systems, especially when synonym queries are sent concurrently to the retriever

Pre-Retrieval

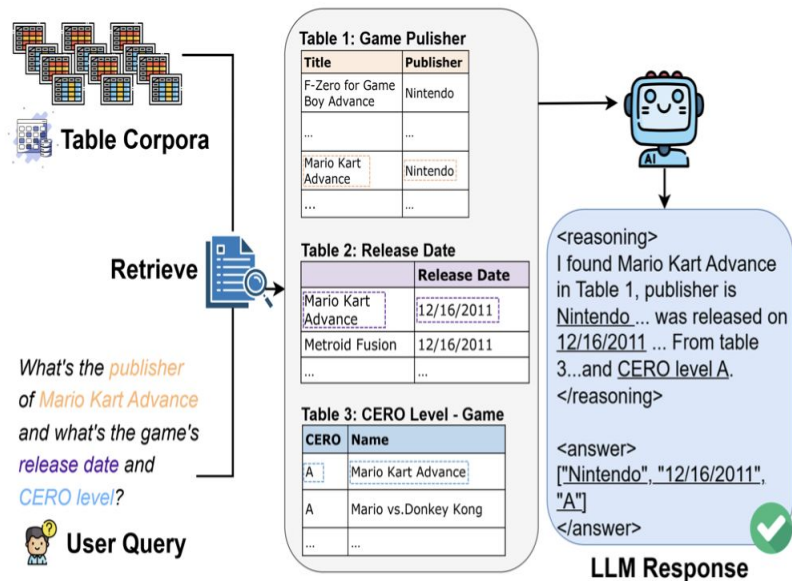
- Main objective is to fix Garbage in/Garbage out
- If the index is poorly structured or query is ambiguous, no amount of retrieval or tuning can save the output
- Your RAG system isn't bad. Your queries are ©
- There are 2 main topics here:
 - **Query Transformation methods (including query expansion)**
 - Index & Corpus Design methods (chunking, metadata, multi-vector index)

Pre-Retrieval: Index & Corpus Design

- This IR field has so many solutions that we cannot even mention them
- We will go over two broad categories:
 - Hierarchical Indexing
 - Multi-representation / Multi-vector indexing

Pre-Retrieval: Hierarchical Indexing

- Brief summaries of entire documents (this is top level)
- More detailed summaries of subsections (this is middle level)
- Detailed Chunks
 - Retrieval becomes an efficient, top-down search
 - The query first matches against the high-level summaries
 - Once a relevant summary is found, the system "zooms in" to retrieve the detailed chunks associated only with that summary, ensuring the final context is both relevant and complete



Pre-Retrieval: Multi-representation indexing

- This technique creates multiple vector representations for a single document or chunk. For instance, a single text chunk might be stored with
 - With the text
 - With the summary of the text
 - With the “potential questions” that could be asked about this text (this works really really well)
- Again, this works **really** well in real-time QnA systems

Three Phases of Advanced RAG

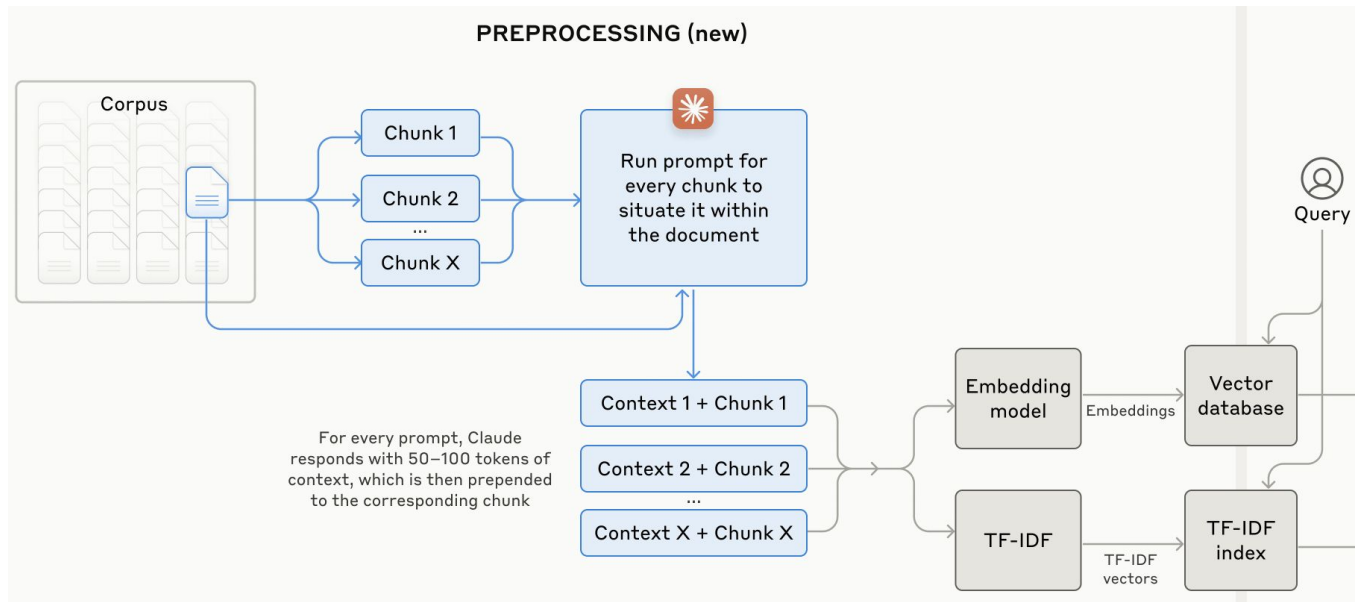
- Modern “Advanced RAG” treats the pipeline as three optimizable stages:
 - Pre-Retrieval (indexing, advanced chunking)
 - **Retrieval (actual retrieval)**
 - Post-Retrieval (compression, filtering, reranking)
-

Retrieval

- The actual Retrieval is a simple step when we compute similarity with docs and incoming query
- However there exist 2 methods that can greatly improve the quality of retrieval system itself:
 - Hybrid Search
 - Fusion Search

Retrieval: Hybrid Search

- Remember BM25? Well, last year Anthropic published infamous post that they actually combined BM25 + semantic similarity RAG



Retrieval: Hybrid Search

- Summary of what they found:
 - Embeddings+BM25 is better than embeddings on their own;
 - Passing the top-20 chunks to the model is more effective than just the top-10 or top-5;
 - Adding context to chunks improves retrieval accuracy a lot;
 - Reranking is better than no reranking;
- All these benefits stack: to maximize performance improvements, we can combine contextual embeddings with contextual BM25, plus a reranking step, and adding the 20 chunks to the prompt.
- In production systems hybrid search is used in very many scenarios

Retrieval: Fusion Algorithms

- After we combined the BM25 + some embedding similarity method we have a problem of merging two lists together (called fusion problem)
- The dominant SOTA fusion algorithm is Reciprocal Rank Fusion (RRF)
- RRF completely ignores the original, uncalibrated scores. It bases its new ranking only on the position (rank) of a document in each list

$$RRF_Score(doc) = \sum_i \frac{1}{k + rank_i(doc)}$$

Three Phases of Advanced RAG

- Modern “Advanced RAG” treats the pipeline as three optimizable stages:
 - Pre-Retrieval (indexing, advanced chunking)
 - Retrieval (actual retrieval)
 - **Post-Retrieval (compression, filtering, reranking)**
-

Post-Retrieval

- As we discussed Retrieval is optimized for Recall, however you can have a lot of false positives (low precision)
- Post-Retrieval methods here serve the purpose of increasing the precision, while keeping recall the same
- There are 2 most notable classes here:
 - Reranking methods
 - Context compression methods

Post-Retrieval: Reranking

- Re-ranking takes the top k documents from the retriever and applies a more powerful, but computationally expensive, model to create a new, more accurate ranking
 - Cross-encoders - very expensive. This is full interaction model, it concatenates (query, doc) and passes through whole transformer
 - Late-interaction models - cheaper, they work in 2 steps:
 - During indexing, each document is encoded at token level, this is done once
 - During test-time, the query is encoded at each token and then finds the max similarity between tokens of query and tokens of document



Post-Retrieval: Context Compression

- Compression removes irrelevant sentences or tokens from within the selected documents, "de-noising" the context before it reaches the LLM
- Can be done through some other LLM
- Also can be done through xRAG
- xRAG reinterprets document embeddings in dense retrieval—traditionally used solely for retrieval—as features from the retrieval modality

Conclusion So Far

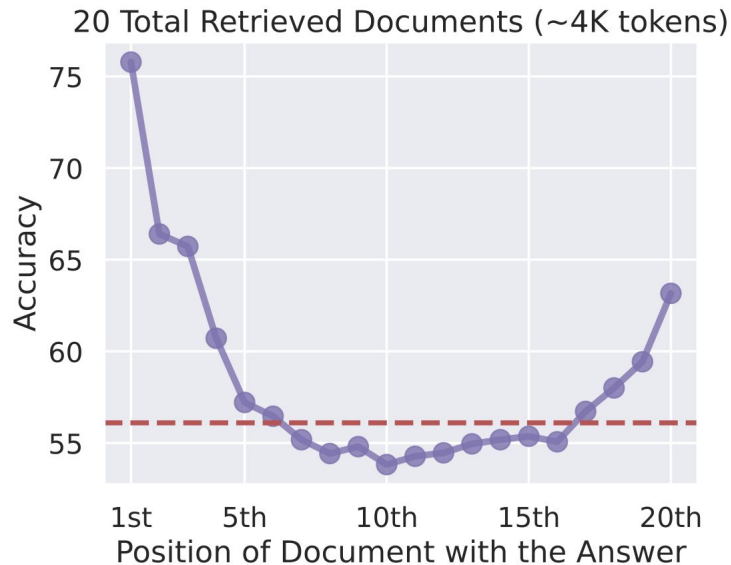
- Because of the “black-boxiness of LLMs” if you are really working on something the change is that improving of the Retriever becomes more like an engineering task, instead of the optimization task

Long Context LLMs vs RAG

- Last year with the appearances of Long Context Windows LLMs there were claims that RAG is dead
- Is that really the case?
- Long Context LLM and RAG serve same goal:
 - to ground the response based on the context
 - to reduce hallucinations
- However there is very distinct difference between two

Lost in the Middle problem

- TLDR: The position **where** you put the document in LLM matters for the quality of responses



Long Context LLMs vs RAG

- Last year with the appearances of Long Context Windows LLMs there were claims that RAG is dead
- Is that really the case?
- Long Context LLM and RAG serve same goal:
 - to ground the response based on the context
 - to reduce hallucinations
- However there is very distinct difference between two

Plan

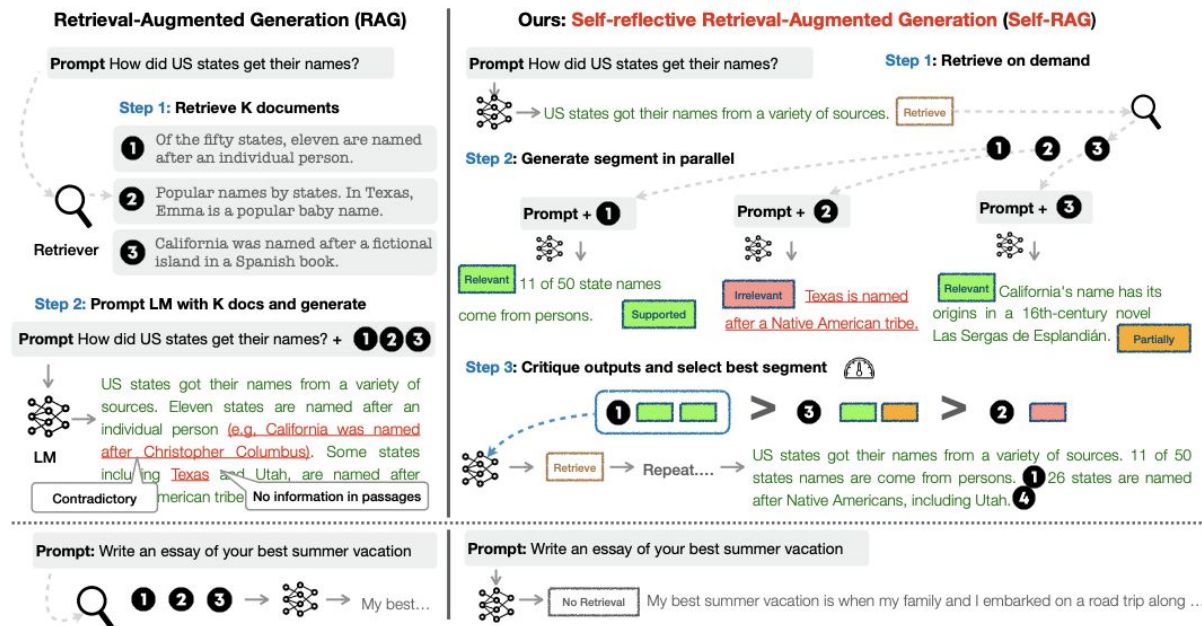
- LLMs are perfect hallucinators
- Information Retrieval and Relevance Gap
- Naive RAG
- How to train Naive RAG
- Advanced RAG
- **Iterative RAG**
- Rag is Dead ©
- Future of retrieval and agentic RAG

Iterative RAG

- The idea in this class of methods is to move beyond linear “retrieve-then-generate” and do iterations on it
- We will go over 3 notable versions of it:
 - Self-RAG
 - GraphRAG
 - Agentic RAG

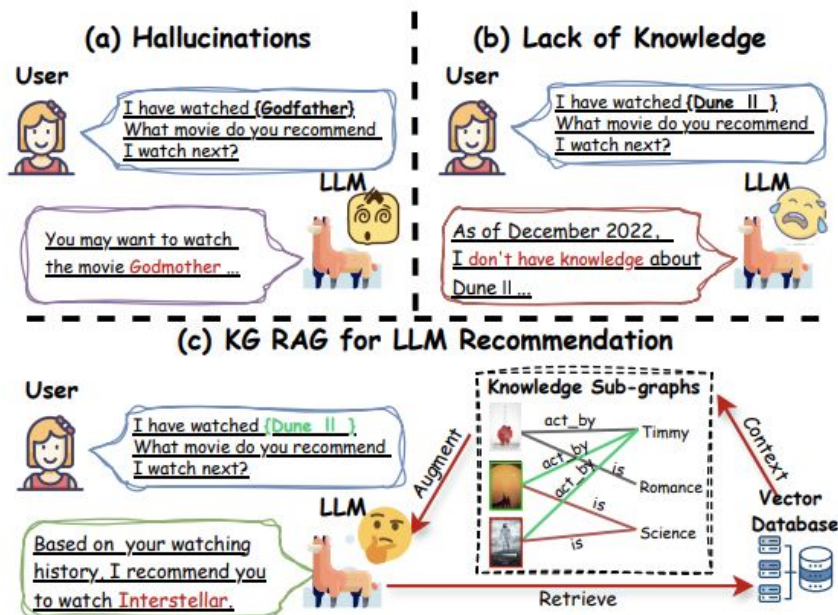
Self-RAG (Asai et al., 2023)

- Trains a single LLM to control its own RAG process through self-reflection



Graph RAG (Wang et al., 2023)

- Vector-search-based cannot "connect the dots" between disparate pieces of information



Graph RAG (Wang et al., 2023)

- The main idea is to use KG (Knowledge Graph) instead of plain vector
- At query time, the system retrieves sub-graphs and semantic communities from this KG, not just isolated text chunks
- Structured context allows GraphRAG to answer complex questions that are impossible for vector search

The main advantage of this method is the out-of-the-box multi-hop reasoning

Agentic RAG (Wang et al., 2023)

- Agentic RAG embeds the entire RAG pipeline within an autonomous AI agent framework
- We will talk more about Agents next lecture but the main advantage is the idea of Self-RAG on steroids:
 - Agent can plan what to retrieve next
 - Agent can use tools in order to get more information
 - Agent can call evaluation to determine current performance

Evaluation of RAG-based systems

- Classical IR metrics are used, however new more Gen AI metrics are also being used:
 - Faithfulness / Groundedness
 - Answer Relevance
- Evaluation frameworks that work explicitly for RAG:
 - RAGAS
 - DeepEval

Future of RAG

- Impossible to tell, but we will try to speculate
- First point: at this current stage RAG is a costyl, it does not fundamentally adress the hallucination problem
- Second point: RAG currently is more about heuristics and engineering/pipelining than about optimization
- Third point: in 2025 ironically people turned back to the 2020 paper on Joint RAG and try to go back to roots to optimize jointly generator + retriever
-